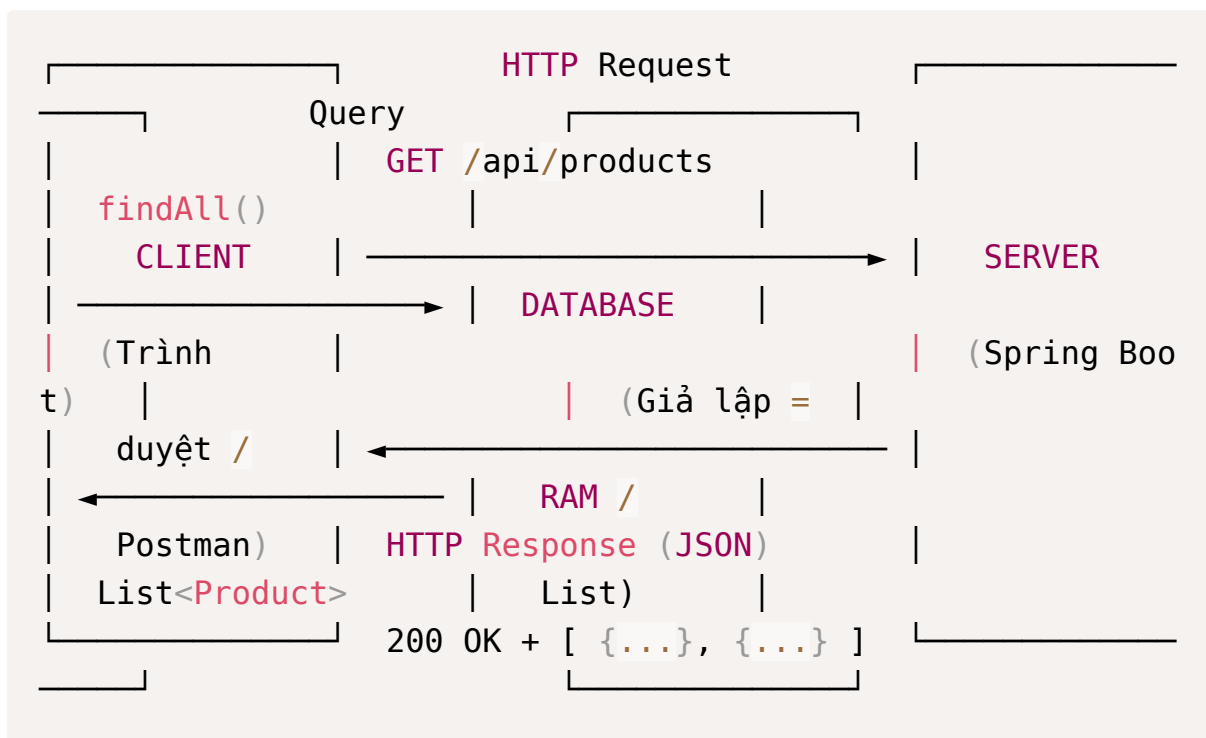


[Bài tập] Phân tích mô hình Client - Server — Ứng dụng Quản lý sản phẩm

Em xin trình bày bài phân tích mô hình Client - Server cho ứng dụng Quản lý sản phẩm như sau ạ.

1. Mô hình tổng quan

Theo em hiểu, mô hình Client - Server là kiến trúc trong đó **Client** (bên yêu cầu) gửi **Request** đến **Server** (bên xử lý), Server xử lý và trả về **Response**. Client và Server giao tiếp qua giao thức **HTTP/HTTPS**.



2. Phân tích chi tiết yêu cầu "Xem danh sách sản phẩm"

2.1. Client — Trình duyệt gửi Request như thế nào?

Theo em, khi người dùng muốn xem danh sách sản phẩm, Client sẽ gửi một request "lấy dữ liệu về". Vì đây là thao tác **đọc / truy vấn dữ liệu** (không tạo,

không thay đổi), nên em chọn dùng HTTP Method là **GET**.

Thành phần	Giá trị dự kiến	Giải thích
URL	<code>http://localhost:8080/api/products</code>	Endpoint đại diện cho tài nguyên "products" (danh sách sản phẩm)
HTTP Method	<code>GET</code>	Dùng để lấy / đọc dữ liệu. Không dùng POST vì không tạo mới dữ liệu
Headers	<code>Accept: application/json</code>	Client báo cho Server biết muốn nhận dữ liệu dạng JSON
Body	<i>(rỗng)</i>	Request GET thường không có body

Vì sao GET mà không phải POST?

GET = đọc dữ liệu (an toàn, idempotent, có thể cache). POST = tạo mới dữ liệu (gây thay đổi trên Server). Vì "xem danh sách" chỉ đọc nên **GET** là đúng chuẩn RESTful.

3.2. Server — Spring Boot làm gì khi nhận yêu cầu?

Theo em phân tích, Server (ứng dụng Spring Boot) nhận request và xử lý qua các tầng (layers):

1. **Controller** nhận request tại URL `/api/products` với method `GET`, thông qua annotation `@GetMapping`.
2. **Controller** gọi xuống tầng **Service** để xử lý logic nghiệp vụ.
3. **Service** gọi tầng **Repository / Data** để lấy dữ liệu (ở bài này giả lập bằng bộ nhớ RAM — một `List<Product>`).
4. Server đóng gói kết quả và trả **Response** về cho Client dưới dạng **JSON**.

```
@RestController
@RequestMapping("/api/products")
public class ProductController {

    // Giả lập Database bằng bộ nhớ RAM (List trong memory)
    private final List<Product> products = new ArrayList<>()
    (List.of(
```

```

        new Product(1, "Bàn phím cơ", 850000),
        new Product(2, "Chuột không dây", 350000),
        new Product(3, "Màn hình 24 inch", 2500000)
    ));

    // Client gọi: GET http://localhost:8080/api/products
    @GetMapping
    public List<Product> getAllProducts() {
        return products; // Spring Boot tự động chuyển List
        -> JSON
    }
}

```

3.3. Database — Đóng vai trò gì?

Em hiểu Database là nơi **lưu trữ dữ liệu** sản phẩm. Trong yêu cầu này, Database chịu trách nhiệm **cung cấp danh sách sản phẩm** khi Server truy vấn.

Tạm thời giả lập bằng bộ nhớ RAM: thay vì kết nối PostgreSQL/MySQL thật, ta dùng một `List<Product>` khai báo sẵn trong Java. Điều này giúp tập trung hiểu **luồng Request - Response** trước, sau đó mới thay bằng database thật (JPA/Repository) ở bước sau.

Giai đoạn	Cách lưu trữ	Cách Server truy vấn
Hiện tại (giả lập)	<code>List<Product></code> trong RAM	Trả trực tiếp list <code>products</code>
Về sau (thật)	Bảng <code>products</code> trong PostgreSQL	<code>productRepository.findAll()</code>

3.4. Response — Server trả về định dạng gì?

Ở bài này em chọn ứng dụng kiểu **REST API** (Spring Boot dùng `@RestController`), nên Server trả về dữ liệu dạng **JSON** — không phải HTML. JSON gọn nhẹ, dễ cho Frontend (React/Next.js) hoặc Postman xử lý.

HTTP/1.1 200 OK

Content-Type: application/json

```

[
  { "id": 1, "name": "Bàn phím cơ", "price": 850000 },
  { "id": 2, "name": "Chuột không dây", "price": 350000 },

```

```
[ { "id": 3, "name": "Màn hình 24 inch", "price": 2500000 } ]
```

JSON hay HTML?

- **JSON** → khi Server là REST API, Frontend tự render giao diện (kiến trúc tách rời — phù hợp React/Next.js).
- **HTML** → khi Server render sẵn trang (dùng Thymeleaf/JSP - kiến trúc truyền thống).

Bài này dùng **JSON** vì định hướng REST API + Frontend riêng.

4. Tổng kết quy trình Request - Response

Bước	Thành phần	Hành động
1	Client	Người dùng bấm "Xem sản phẩm" → gửi <code>GET /api/products</code>
2	Server (Controller)	<code>@GetMapping</code> bắt request tại <code>/api/products</code>
3	Server (Service)	Xử lý logic, yêu cầu lấy dữ liệu
4	Database (RAM)	Trả về <code>List<Product></code>
5	Server	Chuyển <code>List<Product></code> → JSON
6	Client	Nhận <code>200 OK</code> • JSON, hiển thị danh sách sản phẩm

Trên đây là phần phân tích của em về mô hình Client - Server áp dụng cho chức năng "Xem danh sách sản phẩm". Em rất mong nhận được góp ý của thầy/cô để hoàn thiện hơn ạ.